

IngresoQR

Sistema Multi-Tenant de Control de Acceso
Documentacion Tecnica Completa



Version 2.0 | Febrero 2026
FastAPI + React + MongoDB + Raspberry Pi

Indice de Contenido

- 1. Vision General del Sistema
- 2. Arquitectura y Stack Tecnologico
- 3. Estructura del Proyecto (Arbol de Archivos)
- 4. Roles de Usuario y Permisos
- 5. Esquema de Base de Datos (Colecciones MongoDB)
- 6. Referencia de API del Backend
- 7. Paginas del Frontend y Rutas
- 8. Autenticacion y Seguridad
- 9. Sistema de Codigos QR (Dinamico y Estatico)
- 10. Integracion Raspberry Pi / IoT
- 11. Sistema de Pagos (Stripe y MercadoPago)
- 12. Modelo SaaS Multi-Tenant
- 13. Sistema de Email (SMTP por Gimnasio)
- 14. Tareas en Segundo Plano (Cron)
- 15. Guia de Despliegue (Plesk VPS)
- 16. Variables de Entorno
- 17. Resolucion de Problemas y FAQ

1. Vision General del Sistema

IngresoQR es un sistema SaaS multi-tenant completo para el control de acceso en gimnasios. Proporciona a los propietarios de gimnasios una solucion integral para gestionar membresias, controlar el acceso fisico mediante codigos QR, procesar pagos, hacer seguimiento de asistencia y administrar clases. El sistema consta de tres componentes principales:

- Panel de Administracion: Interfaz web para que los administradores gestionen todos los aspectos del gimnasio
- PWA para Socios: Aplicacion Web Progresiva para que los socios accedan a su QR, reserven clases y vean estadisticas
- Controlador Raspberry Pi: Dispositivo IoT que lee codigos QR y controla los torniquetes de entrada/salida mediante rele GPIO

Funcionalidades Principales

- Codigos QR dinamicos que se refrescan cada 5/10/15 segundos (configurable por gimnasio)
- Modo QR estatico para socios con problemas de conectividad
- Anti-passback: Alterna automaticamente entre entrada y salida
- Roles multi-nivel: Super Admin, Gym Admin, Gestor, Entrenador, Socio
- Integracion con Stripe y MercadoPago (claves API por gimnasio)
- Gestion de planes SaaS con 12 flags de funcionalidades configurables
- Programacion de clases con patrones recurrentes y sistema de reservas
- TPV (Punto de Venta) para inventario de productos y ventas
- Gamificacion con insignias y rachas
- Rutinas personalizadas asignadas por entrenadores a socios
- Formularios de registro personalizados por gimnasio
- Sistema de pases de invitados con acceso QR
- Sistema de email SMTP (configurable por gimnasio)
- Dashboard de analiticas con mapas de calor horarios y metricas de retencion
- Exportacion Excel de datos de socios
- Informes de ventas en PDF
- Seguridad: Bloqueo de IP tras intentos fallidos de login
- Gestion de dispositivos de socios (limitar dispositivos por socio)
- Modo kiosko para auto-registro en el gimnasio
- Soporte de tema oscuro/claro

2. Arquitectura y Stack Tecnologico

Backend

- Framework: FastAPI (Python 3.11+)
- Base de datos: MongoDB (via Motor async driver)
- Autenticacion: JWT (python-jose + bcrypt)
- Servidor ASGI: Uvicorn
- Almacenamiento de archivos: Sistema de archivos local (/opt/gymaccess/uploads)
- Email: aiosmtplib (SMTP asincrono)
- Pagos: Stripe SDK, MercadoPago SDK
- Excel: openpyxl

Frontend

- Framework: React 18 (Create React App)
- Estilos: Tailwind CSS + variables CSS personalizadas para temas
- Componentes UI: Shadcn/UI (basados en Radix)
- Cliente HTTP: Axios
- Enrutamiento: React Router v6
- Notificaciones: Sonner
- Fuentes: Chivo (titulos) + Manrope (cuerpo)
- Iconos: Lucide React

IoT (Raspberry Pi)

- Hardware: Raspberry Pi 3B+ con modulo de 2 reles
- Escaner QR: Escaner USB de codigos de barras/QR (emulacion de teclado HID)
- GPIO: Modo BCM, Pin 12 (rele entrada), Pin 16 (rele salida)
- Librerias: RPi.GPIO, evdev, requests, python-dotenv

Flujo de Comunicacion

1. Socio abre la PWA -> Frontend solicita QR al backend -> Backend genera QR firmado con HMAC
2. Socio muestra QR en el torniquete -> Escaner del Raspberry Pi lee el QR
3. Raspberry Pi envia QR a POST /api/access/validate
4. Backend valida firma, verifica membresia, registra log de acceso
5. Backend responde valido/invalido -> Raspberry Pi activa rele de entrada o salida por 3 segundos

3. Estructura del Proyecto

```
/app
|-- backend/
|   |-- server.py           # App FastAPI, routers, tareas inicio
|   |-- models.py          # Modelos Pydantic de peticiones
|   |-- auth.py             # JWT, hash passwords, checks de rol
|   |-- database.py         # Conexion MongoDB (Motor)
|   |-- qr_utils.py         # Generacion y validacion QR (HMAC)
|   |-- storage.py          # Helper almacenamiento local
|   |-- requirements.txt    # Dependencias Python
|   |-- .env                # Variables de entorno
|   |-- routes/
|       |-- auth_routes.py  # Login, registro, perfil
|       |-- gym_routes.py   # CRUD gimnasios, capacidad
|       |-- member_routes.py # CRUD socios, exportar Excel
|       |-- plan_routes.py  # Planes de membresia
|       |-- access_routes.py # QR, validar, logs acceso
|       |-- device_routes.py # Registro dispositivos IoT
|       |-- payment_routes.py # Stripe checkout, webhooks
|       |-- mercadopago_routes.py # Integracion MercadoPago
|       |-- class_routes.py  # Clases, horarios, reservas
|       |-- saas_routes.py   # Planes SaaS, suscripciones
|       |-- pos_routes.py    # Productos, ventas, TPV
|       |-- accounting_routes.py # Informes financieros, PDF
|       |-- analytics_routes.py # Mapas calor, retencion
|       |-- notification_guest_routes.py # Notificaciones e invitados
|       |-- security_routes.py # Bloqueo IP, intentos login
|       |-- upload_routes.py  # Avatar, logo, imagenes
|       |-- form_routes.py    # Formularios personalizados
|       |-- gamification_routes.py # Insignias y rachas
|       |-- routine_routes.py # Rutinas de entrenamiento
|       |-- misc_routes.py    # Email, dashboard, descargas
|   |-- downloads/          # Scripts descargables
|   |-- tests/              # Tests con Pytest
|-- frontend/
|   |-- src/
|       |-- App.js          # Router principal
|       |-- App.css         # Estilos globales + variables CSS
|       |-- context/AuthContext.js # Estado de autenticacion
|       |-- lib/api.js       # Funciones API con Axios
|       |-- lib/utils.js     # Funciones utilitarias
|       |-- components/      # Componentes reutilizables
|       |-- layouts/         # Layouts Admin y PWA
|       |-- pages/admin/     # 28 paginas del panel admin
|       |-- pages/pwa/       # 14 paginas de la PWA socio
|   |-- build/              # Build de produccion
```

4. Roles de Usuario y Permisos

Rol	gym_id	Descripcion
super_admin	null	Control total de la plataforma. Gestiona gimnasios, planes SaaS, dispositivos.
gym_admin	UUID	Control total de su gimnasio. Socios, planes, pagos, clases, config.
gym_manager	UUID	Permisos configurables. Subconjunto de gym_admin.
trainer	UUID	Gestiona clases asignadas. Crea rutinas. Dashboard propio.
member	UUID	Accede a la PWA. Ve QR, reserva clases, perfil.

Permisos del Gestor (Configurables)

Clave Permiso	Descripcion
members_view	Ver lista de socios
members_create	Crear nuevos socios
members_edit	Editar datos de socios
members_delete	Eliminar socios
members_suspend	Suspender/Reactivar socios
payments_register	Registrar pagos manuales
pos_sell	Procesar ventas TPV
pos_products	Gestionar productos TPV
access_view	Ver logs de acceso
classes_manage	Gestionar clases y horarios
data_export	Exportar datos (Excel)
notifications_send	Enviar notificaciones

5. Esquema de Base de Datos (MongoDB)

MongoDB es la base de datos principal. Todos los documentos usan un campo 'id' UUID string como clave primaria (no ObjectId). El campo '_id' siempre se excluye de las respuestas API.

Coleccion: admins

Usuarios administradores (super_admin, gym_admin, gym_manager, trainer)

```
id, email, password (bcrypt), name, role, gym_id, permissions[], active, created_at
```

Coleccion: gyms

Gimnasios (tenants)

```
id, name, address, phone, email, logo_url, primary_color, qr_refresh_seconds, qr_mode, max_members, api_token, stripe_secret_key, currency, smtp_*, saas_plan_id, status, created_at
```

Coleccion: members

Socios del gimnasio

```
id, email, name, phone, code (6 chars), gym_id, status, gender, avatar_path, qr_mode, suspension_reason, can_bring_guests, created_at
```

Coleccion: plans

Planes de membresia

```
id, gym_id, name, description, price, duration_days, access_type, active, created_at
```

Coleccion: memberships

Membresias activas/vencidas

```
id, member_id, plan_id, gym_id, start_date, end_date, status (active/expired), payment_id, created_at
```

Coleccion: access_logs

Registros de entrada/salida

```
id, member_id, member_name, member_code, gym_id, direction (entrada/salida), timestamp, is_guest
```

Coleccion: payment_transactions

Registros de pagos

```
id, session_id, member_id, plan_id, gym_id, amount, currency, status, payment_method, created_at
```

Coleccion: classes

Definiciones de clases

```
id, gym_id, name, trainer_id, max_capacity, duration_minutes, recurring, days_of_week[], start_time, end_date, active
```

Coleccion: class_schedules

Horarios individuales de clases

```
id, class_id, gym_id, date, start_time, end_time, max_capacity, current_bookings, status
```

Coleccion: bookings

Reservas de clases por socios

```
id, member_id, schedule_id, class_id, gym_id, date, status, checked_in, checked_in_at
```

Coleccion: devices

Dispositivos IoT registrados

```
id, gym_id, name, location, status, last_ping, created_at
```

Coleccion: notifications

Notificaciones del gimnasio para socios

```
id, gym_id, title, message, target, read_by[], created_at
```

Coleccion: guests

Pases de invitados

```
id, code, name, invited_by, gym_id, valid_until, status, accesses, created_at
```

Coleccion: saas_plans

Planes SaaS de la plataforma

```
id, name, max_members, has_qr_access, has_classes, has_pos, ... (12 flags), price_monthly, currency, active
```

Coleccion: pos_products

Inventario de productos TPV

```
id, gym_id, name, cost_price, sale_price, stock, category, active
```

Coleccion: pos_sales

Transacciones de ventas TPV

```
id, gym_id, items[], total, payment_method, registered_by, created_at
```

Coleccion: email_logs

Historial de emails enviados

```
id, gym_id, member_id, to_email, subject, status, sent_at
```

Coleccion: routines

Rutinas de entrenamiento

```
id, gym_id, trainer_id, member_id, name, days[], active
```

Coleccion: broadcasts

Anuncios de la plataforma (Super Admin)

```
id, title, message, priority, active, dismissed_by[]
```

Coleccion: blocked_ips

IPs bloqueadas por fuerza bruta

```
ip, blocked_until, reason, blocked_at
```

6. Referencia de API del Backend

Todos los endpoints estan prefijados con /api. La autenticacion es via Bearer JWT token en el header Authorization, excepto los marcados como publicos.

Autenticacion

Metodo	Endpoint	Descripcion
POST	/api/auth/admin/register	Registrar nuevo admin
POST	/api/auth/admin/login	Login admin, devuelve JWT
POST	/api/auth/admin/impersonate/{gym_id}	Super Admin suplanta Gym Admin
PUT	/api/auth/admin/update-profile	Actualizar email/password
POST	/api/auth/member/login?code=XXX	Login socio por codigo
GET	/api/auth/member/me	Perfil socio actual + gym + membresia

Gimnasios

Metodo	Endpoint	Descripcion
POST	/api/gyms	Crear gimnasio (+ admin opcional)
GET	/api/gyms	Listar gimnasios (filtrado por rol)
PUT	/api/gyms/{id}	Actualizar configuracion
DELETE	/api/gyms/{id}	Eliminar gimnasio + datos
PUT	/api/gyms/{id}/suspend	Suspender/Reactivar
GET	/api/gyms/{id}/capacity	Stats de capacidad
GET	/api/gyms/{id}/public-info	Publico: nombre, logo, pagos

Socios

Metodo	Endpoint	Descripcion
POST	/api/members	Crear socio (admin)
POST	/api/members/register	Auto-registro publico
GET	/api/members	Listar socios
PUT	/api/members/{id}	Actualizar socio
POST	/api/members/{id}/suspend	Suspender con motivo
DELETE	/api/members/{id}	Eliminar socio
GET	/api/members/export/excel	Exportar a .xlsx

Planes y Membresias

Metodo	Endpoint	Descripcion
POST	/api/plans	Crear plan
GET	/api/plans	Listar planes
POST	/api/memberships	Asignar membresia
GET	/api/memberships/expiring	Membresias por vencer

Acceso y QR

Metodo	Endpoint	Descripcion
GET	/api/qrcode/generate	Generar QR del socio logueado
POST	/api/access/validate	PUBLICO: Validar QR (Raspberry Pi)
GET	/api/access/logs	Logs de acceso
GET	/api/access/stats	Estadisticas de acceso
GET	/api/access/stats/daily	Accesos diarios
GET	/api/access/stats/hourly	Distribucion horaria

Pagos

Metodo	Endpoint	Descripcion
POST	/api/payments/checkout	Crear sesion Stripe
GET	/api/payments/status/{session_id}	Estado del pago
POST	/api/payments/manual	Registrar pago manual
GET	/api/payments/history	Historial de pagos
PUT	/api/gyms/{id}/stripe-config	Actualizar clave Stripe

Clases y Reservas

Metodo	Endpoint	Descripcion
POST	/api/classes	Crear clase
GET	/api/classes	Listar clases
PUT	/api/classes/{id}	Editar clase (regenera horarios)
DELETE	/api/classes/{id}	Eliminar clase + horarios
POST	/api/bookings	Reservar clase (socio)
POST	/api/bookings/{id}/checkin	Check-in de asistencia
POST	/api/classes/cleanup-stale-schedules	Limpiar horarios huérfanos

Gestion SaaS

Metodo	Endpoint	Descripcion
POST	/api/saas/plans	Crear plan SaaS
PUT	/api/saas/plans/{id}	Editar plan SaaS
PUT	/api/gyms/{id}/saas-plan	Asignar plan SaaS a gym
GET	/api/gyms/{id}/saas-features	Features activas del gym
POST	/api/saas/subscribe	Suscribirse a plan SaaS

TPV (Punto de Venta)

Metodo	Endpoint	Descripcion
POST	/api/pos/products	Crear producto
POST	/api/pos/sales	Registrar venta
GET	/api/pos/stats	Estadisticas TPV

Seguridad

Metodo	Endpoint	Descripcion
GET	/api/security/blocked-ips	IPs bloqueadas
DELETE	/api/security/blocked-ips/{ip}	Desbloquear IP
GET	/api/security/login-attempts	Intentos de login

7. Paginas del Frontend y Rutas

Panel Admin (/admin/*)

Ruta	Componente	Descripcion
/admin/login	AdminLogin.js	Formulario de login admin
/admin	AdminDashboard.js	Dashboard principal con estadisticas
/admin/gyms	AdminGyms.js	Gestion de gimnasios
/admin/members	AdminMembers.js	Gestion de socios
/admin/plans	AdminPlans.js	Planes de membresia
/admin/classes	AdminClasses.js	Gestion de clases
/admin/attendance	AdminAttendance.js	Asistencia a clases
/admin/schedules	AdminSchedules.js	Calendario de horarios
/admin/staff	AdminStaff.js	Gestion de personal
/admin/access	AdminAccess.js	Visor de logs de acceso
/admin/devices	AdminDevices.js	Gestion dispositivos IoT
/admin/notifications	AdminNotifications.js	Notificaciones
/admin/guests	AdminGuests.js	Pases de invitados
/admin/accounting	AdminAccounting.js	Informes financieros
/admin/pos	AdminPOS.js	Punto de Venta
/admin/analytics	AdminAnalytics.js	Analiticas avanzadas
/admin/saas-plans	AdminSaaSPlans.js	Config planes SaaS
/admin/settings	AdminSettings.js	Configuracion del gym
/admin/security	AdminSecurity.js	Dashboard de seguridad

PWA Socios (/app/*)

Ruta	Componente	Descripcion
/app/login	MemberLogin.js	Login con codigo de socio
/app	MemberHome.js	QR + acceso rapido
/app/classes	MemberClasses.js	Reservar clases
/app/history	MemberHistory.js	Historial de accesos
/app/stats	MemberStats.js	Estadisticas personales
/app/membership	MemberMembership.js	Detalles membresia + pagar
/app/guests	MemberGuests.js	Invitar invitados
/app/achievements	MemberGamification.js	Insignias y rachas

/app/routines	MemberRoutines.js	Rutinas de entrenamiento
/app/profile	MemberProfile.js	Configuracion de perfil

8. Autenticacion y Seguridad

Autenticacion JWT

Se usan tokens JWT con algoritmo HS256. Los tokens expiran tras 24 horas. El payload contiene: sub (ID usuario), role, gym_id.

Bloqueo de IP (Proteccion Fuerza Bruta)

Tras 5 intentos fallidos desde la misma IP en 15 minutos, la IP se bloquea 15 minutos. Aplica a login de admin y socio. El Super Admin puede desbloquear desde el dashboard de Seguridad.

Suplantacion

El Super Admin puede suplantar a cualquier Gym Admin via POST /api/auth/admin/impersonate/{gym_id}. Esto crea un JWT especial con impersonating=true.

9. Sistema de Codigos QR

QR Dinamico

Los QR dinamicos se regeneran cada N segundos (configurable: 5, 10 o 15). Cada QR contiene: member_id, gym_id, timestamp Unix, firma HMAC-SHA256 (16 chars hex). Los datos se codifican en base64url.

QR Estatico

Para socios con mala conectividad, el Super Admin puede asignar modo QR estatico. Estos QR no expiran (sin verificacion de timestamp).

Anti-Passback

El sistema implementa anti-passback automatico verificando el ultimo log de acceso del socio. Si el ultimo fue 'entrada', el siguiente sera 'salida' y viceversa.

10. Integración Raspberry Pi

Requisitos de Hardware

- Raspberry Pi 3B+ o superior
- Modulo de 2 rele (5V)
- Escaner QR USB (modo HID - actua como teclado)
- Cableado: GPIO 12 -> Rele entrada, GPIO 16 -> Rele salida

Configuracion de Software

```
# /opt/gymaccess/.env
GYMACCESS_SERVER_URL=https://c.ingresoqr.com
GYMACCESS_GYM_TOKEN=tu_token_del_gimnasio
GYMACCESS_DEVICE_ID=tu_device_uuid
GYMACCESS_RELAY_ENTRADA=12
GYMACCESS_RELAY_SALIDA=16
```

Flujo de Operacion

1. El script usa evdev para leer escaners QR USB como dispositivos de entrada
2. Se pueden configurar multiples escaners via scanner_map.json
3. Al escanear un QR, envia POST a /api/access/validate
4. Si es valido: activa el rele correspondiente por 3 segundos
5. Ping de heartbeat cada 60 segundos a /api/devices/{id}/ping

11. Sistema de Pagos

Stripe

Cada gimnasio puede configurar su propia clave secreta de Stripe. Si no tiene, se usa la variable `STRIPE_API_KEY` global. Los pagos usan Stripe Checkout Sessions.

MercadoPago

Cada gimnasio puede configurar su propio access token de MercadoPago para mercados latinoamericanos.

Pagos Manuales

Los admins pueden registrar pagos manuales (efectivo, tarjeta, transferencia) que activan inmediatamente la membresia.

12. Modelo SaaS Multi-Tenant

La plataforma opera como SaaS donde cada gimnasio es un tenant. El Super Admin crea planes SaaS con flags de funcionalidades configurables. Cada gimnasio se asigna a un plan que determina sus funcionalidades.

Feature Flags (12 toggles)

Clave	Descripcion
has_qr_access	Control de acceso QR
has_guest_passes	Sistema de invitados
has_classes	Programacion de clases y reservas
has_pos	Punto de Venta (TPV)
has_analytics	Dashboard de analiticas avanzadas
has_gamification	Sistema de insignias y rachas
has_routines	Rutinas de entrenamiento
has_email_smtp	Envio de emails SMTP personalizado
has_stripe_members	Pagos Stripe para socios
has_mercadopago	Pagos MercadoPago
has_iframes	Widgets embebibles (iframes)
has_advanced_accounting	Informes contables avanzados

13. Sistema de Email

Cada gimnasio configura su propio servidor SMTP en Configuracion. El sistema envia: emails de bienvenida, recordatorios de vencimiento (1, 3, 7 dias), confirmaciones de pago, y emails con plantillas editables.

14. Tareas en Segundo Plano

Dos tareas ejecutan diariamente a medianoche UTC via asyncio:

- Auto-Suspender: Verifica membresias activas vencidas y suspende socios.
- Recordatorios: Envia emails a socios con membresias por vencer en 1, 3 o 7 dias.

15. Guia de Despliegue (Plesk VPS)

Backend (c.ingresoqr.com)

```
# 1. SSH al servidor
ssh root@tu-servidor

# 2. Instalar dependencias
apt install python3.11 python3.11-venv mongodb-org

# 3. Crear directorio de la app
mkdir -p /opt/gymaccess && cd /opt/gymaccess

# 4. Entorno virtual
python3.11 -m venv venv
source venv/bin/activate
pip install -r requirements-prod.txt

# 5. Configurar .env (MONGO_URL, JWT_SECRET, QR_SECRET)

# 6. Ejecutar
nohup venv/bin/uvicorn server:app --host 0.0.0.0 --port 8001 &

# 7. Configurar proxy Nginx/Plesk:
#   c.ingresoqr.com -> proxy_pass http://127.0.0.1:8001
```

Frontend (app.ingresoqr.com)

El comando git clone SOLO actualiza archivos del backend (.py). Para cambios en el frontend, debes reconstruir (yarn build) y subir manualmente la carpeta build/ a Plesk.

16. Variables de Entorno

Variable	Descripcion
MONGO_URL	Cadena de conexion MongoDB
DB_NAME	Nombre de la base de datos
JWT_SECRET	Secreto para firma JWT
QR_SECRET	Secreto para firma HMAC de QR
STRIPE_API_KEY	Clave Stripe global (fallback)
PLATFORM_STRIPE_KEY	Clave Stripe para pagos SaaS
UPLOAD_DIR	Directorio de subida de archivos
REACT_APP_BACKEND_URL	URL del backend (frontend)

17. Resolucion de Problemas

Error 'Algo salio mal' en el navegador

Es el ErrorBoundary capturando un crash de React. Revisa la consola del navegador (F12) para el error real.

Cambios del frontend no aparecen en produccion

El comando git clone SOLO actualiza archivos Python del backend. Debes reconstruir el frontend (yarn build) y subir la carpeta build/ manualmente a Plesk.

QR no valida en el torniquete

Verificar: 1) QR_SECRET coincide 2) Sincronizacion de hora 3) Usar /api/access/debug-qr para diagnosticar

Socio suspendido automaticamente

El cron diario auto-suspende socios con membresias vencidas. Renovar membresia para reactivar.